

Skill 设计与开发

把提示词打包成"技能"——让你的 AI 编程助手越用越懂你

 = 该术语在 [01_AI_terminology_explanations.md](#) 中有独立词条，可对照阅读

 **本章定位：**学完第 5 章提示词工程后，你已经会"跟 AI 说话"了。本章要把这些说话技巧**打包成可复用的 Skill**，让 AI 每次都能按你的套路来。这是从"单次提示词"到"Agent 自主决策"之间的关键一步。

目录

一、为什么需要 Skill

- [一、为什么需要 Skill](#) — 场景引入 + 能力阶梯

二、Skill 是什么

- [二、Skill 是什么](#) — 定义 + 切菜比喻 + 结构解剖

三、先用起来：体验 3 个真实 Skill

- [三、先用起来](#) — brainstorming / code-review / verification

四、拆开看：SKILL.md 结构解剖

- [四、拆开看](#) — frontmatter / Overview / Core Pattern / description 写法

五、自己写：手把手第一个 Skill

- [五、自己写](#) — 7 步写一个 code-review Skill

六、设计四原则

- [六、设计四原则](#) — 单一场景 / 提示词为核心 / 渐进式 / 可测试

七、进阶技巧

- [七、进阶技巧](#) — CSO / 防偷懒 / 文件组织 / 引用其他 Skill

七-1/2、实战演示：从零开始写一个 Skill 并测试

- [七-1/2、实战演示](#) — 完整的端到端示例
 - [Step 1-2：创建文件](#)
 - [Step 3：编写内容](#)
 - [Step 4-5：加载与测试](#)
 - [Step 6-7：验证与优化](#)

八、课间练习：5 个实战 Skill 挑战

- [八、课间练习](#) — 5 个渐进式练习
 - [练习 A: API 文档生成](#) ★
 - [练习 B: 单元测试生成](#) ★★
 - [练习 C: Bug 分析报告](#) ★★
 - [练习 D: Git Commit 消息](#) ★★★
 - [练习 E: 数据库查询优化](#) ★★★★★

九、面试常见问题

- [九、面试常见问题](#) — 10 个高频面试题 + 标准答案 + 加分回答

附录

- [术语对照索引](#) — 本章术语与手册的对应关系
- [SKILL.md 模板](#) — 可直接复制的模板
- [质量自检清单](#) — 写完 Skill 后对照检查

〇、学完本章你能做什么

- ☒ 给 Claude Code / Cursor 等 AI 编程助手写自定义 Skill
- ☒ 把高频工作流（调试、代码审查、需求分析）沉淀为可复用的 Skill
- ☒ 用提示词工程  的理论指导 Skill 设计，写出真正好用的 Skill

一、为什么需要 Skill

一个你可能遇到过的场景

你用 AI 编程助手写代码。每次调试 bug，你都要重新跟它解释：

"我们项目用的是 Python 3.11 + FastAPI，代码风格用 Black 格式化，错误处理要统一用自定义异常类，日志用 structlog，测试用 pytest....."

解释完一遍，AI 帮你调试完了。下一次有 bug？——**从头再来一遍。**

这就像你公司每个月都来一个新实习生，你每次都要从头教一遍"我们公司的规矩"。累不累？

如果有一本《员工手册》，新实习生来了自己看，你岂不是省心了？

Skill 就是给 AI 写的"员工手册"

你把手册写好放在那里，AI 每次开始工作前自动翻看——不用你重复交代，它就知道该怎么做。

能力阶梯：从提示词到 Agent



"帮我查天气" "每次查天气
按这个格式" "自己判断
何时查天气"

- **提示词**：你跟 AI 说一次，它做一件事——"帮我写封邮件"
- **Skill**：你把"怎么写邮件"的流程写成手册，AI 每次写邮件都自动按你的套路来
- **Agent**：AI 自己判断什么时候该写邮件、写给谁、什么时候不用写

💡 **一句话总结**：Skill 是"你定好规矩，AI 照着做"。它比提示词省力（不用每次重复），比 Agent 简单（不需要 AI 自己判断）。

二、Skill 是什么

定义

Skill 📖 是封装了多个工具或步骤的复合能力——一个 Markdown 文件，包含了提示词、上下文和工具调用规则，让 AI 在特定场景下按固定的流程做事。

切菜比喻

术语手册里有一个很好的类比：

- **Function Calling** 📖 = 你学会了"切菜"这一招
- **Skill** = 你学会了"做一道完整的菜"——洗菜、切菜、炒菜、摆盘这一整套

比如"订机票"是一个 Function，"查天气"是一个 Function，"找酒店"是一个 Function。把这三个打包成一个"旅行规划 Skill"，你跟 AI 说"帮我规划三亚 5 日游"，它一口气全搞定。

Function vs Skill vs MCP 📖

概念	类比	说明
Function Calling 📖	单个工具（一把刀）	AI 能调用一个外部函数
Skill 📖	工具箱 + 操作手册（一整套厨具 + 菜谱）	多个 Function 打包 + 流程编排
MCP 📖	工具箱接口标准（所有厨具统一接口）	统一标准让 AI 即插即用各种工具

一个 Skill 长什么样

一个 Skill 本质上就是一个 `.md` 文件。打开来看，大致是这样的：

```
---
name: code-review
description: Use when reviewing code changes before merging
---

# Code Review

## Overview
```

审查代码时，关注三个维度：正确性 > 可维护性 > 性能。

When to Use

- PR 合并前的代码审查
- 别人请你帮忙看代码

Core Pattern

1. 理解意图：这段代码要解决什么问题？
2. 检查正确性：逻辑对不对？
3. 检查可维护性：命名清晰吗？
4. 检查性能：有没有明显问题？

Common Mistakes

- 只关注风格问题，忽略逻辑错误
- 只说问题，不说亮点

就这么简单——一个 **Markdown 文件** = 一个 **Skill**。不需要写代码，不需要装框架。

三、先用起来：体验 3 个真实 Skill

在自己写之前，先看看别人写的 Skill 长什么样。以下是三个来自 superpowers 插件的真实 Skill 片段：

Skill 1: brainstorming（头脑风暴）

```
---
name: brainstorming
description: Use before any creative work - creating features,
  building components, adding functionality, or modifying behavior
---

# Brainstorming

## Overview
Help turn ideas into fully formed designs through
natural collaborative dialogue.

## When to Use
- 开始任何新功能开发之前
- 用户说了"让我们构建 x"
- 需要探索需求、理清思路
```

注意：description 只写了"什么时候用"（before any creative work），没写"这个 Skill 能做什么"。

Skill 2: code-review（代码审查）

```
---
name: code-review
description: Review the current diff for correctness bugs
```

```
and reuse/simplification/efficiency cleanups
---
```

注意：description 说的是"审查当前 diff"这个触发场景，不是"我会帮你审查代码"。

Skill 3: verification-before-completion（完成前验证）

```
---
name: verification-before-completion
description: Run BEFORE marking any task complete
---
```

注意：description 极其简洁——"在标记任务完成之前运行"。一句话说清触发条件。

 **发现规律了吗？** 好的 description 都是"Use when..."开头——告诉 AI 什么情况下应该加载这个 Skill。

四、拆开看：SKILL.md 结构解剖

一个完整的 SKILL.md 由以下部分组成：



逐段讲解

① Frontmatter（YAML 头部）

```
---
name: your-skill-name
```

```
description: Use when [specific triggering conditions]
---
```

- **name**：只用字母、数字、连字符（不要括号、特殊字符）
- **description**：只写触发条件，不写做什么（这是最重要的规则！）

② Overview（概述）

核心原则，1-2 句话。让 AI 一眼知道这个 Skill 的"灵魂"是什么。

③ When to Use（什么时候用）

列出触发场景。别忘了写"不要用在"哪里——帮 AI 排除误用。

④ Core Pattern（核心模式）

这是 Skill 的主体。步骤要清晰、可操作、有顺序。

⑤ Quick Reference（速查表，可选）

把常用信息做成表格，AI 查起来更快。

⑥ Common Mistakes（常见错误，可选）

告诉 AI 哪些坑不要踩。

⚠️ Description 的写法：最重要的一课

Description 决定了 AI "什么时候会想到用这个 Skill"。写错了，Skill 就等于不存在。

❌ 错误写法	✅ 正确写法	错在哪
description: 帮用户调试代码	description: Use when tests fail or code behaves unexpectedly	不要写"做什么", 要写"什么时候触发"
description: 用于写周报的工具	description: Use when creating weekly status reports every Friday	要写具体的触发场景
description: For debugging	description: Use when tests fail, performance degrades, or behavior differs from expectations	太抽象, 要写具体症状
description: 代码审查之间做回顾	description: Use when executing implementation plans with independent tasks	不要概述流程——AI 会偷懒只看 description 不去读正文

💡 **记住**：description 回答的是"AI 什么时候应该加载这个 Skill"，而不是"这个 Skill 能做什么"。

五、自己写：手把手第一个 Skill

现在来写你自己的第一个 Skill。我们做一个"代码审查"Skill。

Step 1: 创建文件

在你的 AI 编程助手的 Skill 目录下创建文件：

```
~/ .claude/skills/code-review/SKILL.md
```

Step 2: 写 frontmatter

```
---
name: code-review
description: Use when reviewing code changes before merging,
  or when asked to check code quality
---
```

Step 3: 写 Overview

```
# Code Review

## Overview
审查代码时，关注三个维度：正确性 > 可维护性 > 性能。
先确保代码是对的，再看是否好维护，最后才考虑性能。
```

Step 4: 写 When to Use

```
## When to Use
- PR 合并前的代码审查
- 别人请你帮忙看代码
- 重构后验证代码质量

## When NOT to Use
- 不要用于安全审计（那是专门的 skill）
- 不要用于性能调优（先看正确性）
```

Step 5: 写 Core Pattern

```
## Core Pattern

### 审查流程
1. **理解意图**：这段代码要解决什么问题？
2. **检查正确性**：逻辑对不对？边界条件有没有？
3. **检查可维护性**：命名清晰吗？结构合理吗？
4. **检查性能**：有没有明显的性能问题？
```

```
### 输出格式
| 严重程度 | 位置 | 问题 | 建议 |
|-----|-----|-----|-----|
| ● 严重 | L42 | ... | ... |
| ● 建议 | L78 | ... | ... |
```

Step 6: 写 Common Mistakes

```
## Common Mistakes
- 只关注风格问题（缩进、分号），忽略逻辑错误
- 一次性提太多意见，让作者无所适从
- 只说问题，不说亮点
```

Step 7: 测试

调用你的 Skill，给它一段有 bug 的代码，看它是否按照你写的流程走。如果 AI 跳过了"理解意图"直接给建议——说明你的 Core Pattern 写得不够强硬，需要加"⚠️ 不要跳过第一步"这样的禁止语句。

💡 **完整文件长这样：**把 Step 2-6 的内容拼在一起，就是一个完整的 SKILL.md。不到 50 行。

六、设计四原则

原则一：单一场景

一个 Skill 解决一类问题，不要做大杂烩。

✗ 太宽泛	✓ 合适
debugging（调试一切）	test-debugging（测试失败调试）
code-helper（代码助手）	code-review（代码审查）

为什么？ Skill 越聚焦，AI 越容易按你的套路来。大而全的 Skill 往往每条规则都不够强，AI 容易"选择性执行"。

原则二：提示词为核心

Skill 的核心是"提示词"，不是代码、不是配置。

把提示词写好，比加花哨的功能更重要。一个好 Skill = 一套好提示词 + 清晰的结构。

💡 **和术语手册的联系：**第 5 章学的提示词工程 📖 技巧（角色设定、Few-shot 📖、CoT 📖），都可以直接用在 Skill 的 Core Pattern 里。

原则三：渐进式复杂度

先写最简版本，跑通了再加内容。

V1: 能跑就行 (30 行)
↓ 发现 AI 总是跳过某一步
V2: 加上禁止语句 (50 行)
↓ 发现 AI 输出格式不稳定
V3: 加上输出格式模板 (80 行)
↓ 需要速查
V4: 加上 Quick Reference (100 行)

不要一开始就写 300 行。 越长的 Skill，AI 越容易"选择性阅读"。

原则四：可测试

写完 Skill 要验证——让 AI 按照 Skill 走一遍，看效果是否符合预期。

测试方法：

1. 准备一个测试场景（比如一段有 bug 的代码）
2. 调用你的 Skill
3. 观察 AI 是否按照你写的流程走
4. 如果 AI 跳过了某些步骤 → 说明 Skill 写得不够明确，需要加强

七、进阶技巧

技巧 1：CSO（让 AI 能找到你的 Skill）

AI 是根据 description 中的关键词来决定"要不要加载这个 Skill"的。所以要在 description 和正文中**埋入 AI 会搜索的关键词**：

- 错误信息："test failed"、"timeout"、"unexpected behavior"
- 症状描述："flaky"、"hanging"、"wrong output"
- 工具名称：pytest、FastAPI、Docker

技巧 2：防 AI 偷懒

AI 有时候会跳过某些步骤。在 Skill 中明确禁止：

⚠️ 禁止行为

- 不要跳过"理解意图"步骤直接给建议
- 不要只看代码风格，忽略逻辑问题
- 不要只说问题，不说亮点

****所有禁止行为都意味着：停下来，从头走流程。****

技巧 3：文件组织

情况	做法
所有内容 < 100 行	全放 SKILL.md 一个文件
有大段参考文档 (> 100 行)	拆到 <code>reference.md</code> ，SKILL.md 里引用
有可复用的代码模板	拆到 <code>template.py</code> 等独立文件

原则：SKILL.md 保持精简，重东西放辅助文件。

技巧 4：引用其他 Skill

如果你的 Skill 依赖另一个 Skill 的能力，直接引用，不要重复写：

****前置要求**：**请先理解 ``systematic-debugging`` Skill 中的调试流程。

七-1/2、实战演示：从零开始写一个 Skill 并测试

💡 这一节是完整的端到端示例。跟着做一遍，你就知道 Skill 是怎么工作的了。

场景：写一个"代码审查"Skill

目标：让 Claude Code 在审查代码时，按照固定流程走，不遗漏关键问题。

Step 1：确定文件位置

Claude Code 的 Skill 存放在以下位置之一：

类型	路径	说明
项目级	<code>.claude/skills/your-skill/SKILL.md</code>	只在当前项目生效
全局级	<code>~/.claude/skills/your-skill/SKILL.md</code>	在所有项目生效

我们用项目级，方便测试和管理。

Step 2：创建文件

在项目根目录下创建：

```
mkdir -p .claude/skills/code-review
touch .claude/skills/code-review/SKILL.md
```

目录结构：

```
your-project/
├── .claude/
│   └── skills/
│       └── code-review/
│           └── SKILL.md      ← 你的 Skill 文件
├── src/
│   └── main.py
└── README.md
```

Step 3: 编写 SKILL.md

打开 **SKILL.md**，写入以下内容：

```
---
name: code-review
description: Use when reviewing code changes before merging,
  or when asked to check code quality
---

# Code Review

## Overview
审查代码时，关注三个维度：**正确性 > 可维护性 > 性能**。
先确保代码是对的，再看是否好维护，最后才考虑性能。

## When to Use
- PR 合并前的代码审查
- 别人请你帮忙看代码
- 重构后验证代码质量

## When NOT to Use
- 不要用于安全审计（那是专门的 skill）
- 不要用于性能调优（先看正确性）

## Core Pattern

### Step 1: 理解意图（⚠️ 必须执行，不可跳过）
先回答：
- 这段代码要解决什么问题？
- 预期的输入和输出是什么？

### Step 2: 检查正确性
- 逻辑对不对？
- 边界条件有没有？
- 错误处理是否完善？

### Step 3: 检查可维护性
- 命名清晰吗？
- 结构合理吗？
```

- 有没有重复代码？

Step 4: 检查性能

- 有没有明显的性能问题？
- 有没有不必要的循环或计算？

Output Format

严重程度	位置	问题	建议
🔴 严重	L42	...	...
🟡 建议	L78	...	...
🟢 亮点	L15	...	继续保持

⚠️ 禁止行为

- 不要跳过"理解意图"步骤直接给建议
- 不要只看代码风格，忽略逻辑问题
- 不要只说问题，不说亮点

****所有禁止行为都意味着：停下来，从头走流程。****

Step 4: 在 Claude Code 中加载

方法 1: 自动加载（推荐）

Claude Code 会自动扫描 `.claude/skills/` 目录，你的 Skill 会自动加载。

验证方法：在 Claude Code 中输入 `/skills`，看是否看到 `code-review`。

```
> /skills

Available Skills:
- code-review: Use when reviewing code changes before merging...
```

方法 2: 手动调用

如果 Claude Code 没有自动加载，可以用 `/skill` 命令手动加载：

```
> /skill code-review
```

Step 5: 测试使用

准备测试代码：

创建一个有问题的 Python 文件 `test_code.py`：

```
def calculate_discount(price, discount):  
    # 没有类型检查  
    # 没有边界条件检查  
    # 没有错误处理  
    return price * discount
```

在 Claude Code 中调用：

```
> 帮我审查一下 test_code.py 这个文件  
  
[或者更明确的触发]  
  
> 这段代码有什么问题？帮我 review 一下
```

预期行为：

Claude Code 应该：

- 1.  先问"这段代码要解决什么问题？"（Step 1: 理解意图）
- 2.  然后检查正确性（Step 2）
- 3.  再检查可维护性（Step 3）
- 4.  最后检查性能（Step 4）
- 5.  按照 Output Format 输出结果

实际输出示例：

```
## Step 1: 理解意图  
  
这段代码的目的是计算折扣后价格。  
- 输入：price（原价）、discount（折扣）  
- 输出：折扣后价格  
  
## Step 2: 检查正确性  
  
| 严重程度 | 位置 | 问题 | 建议 |  
|-----|-----|-----|-----|  
|  严重 | L1 | 没有类型检查 | 添加类型注解：price: float, discount: float |  
|  严重 | L4 | 没有边界条件检查 | discount 应该在 0-1 之间 |  
|  建议 | L1 | 缺少文档字符串 | 添加 docstring 说明函数用途 |  
  
## Step 3: 检查可维护性  
  
- 函数名清晰   
- 结构简单   
  
## Step 4: 检查性能  
  
- 没有明显性能问题 
```

- ```
🟢 亮点
```
- 函数逻辑简洁

## Step 6: 验证效果

### 检查 AI 是否按流程走：

- ❌ **失败**：AI 直接给出建议，跳过了"理解意图"步骤
- ✅ **成功**：AI 先问"这段代码要解决什么问题？"，然后按步骤走

### 如果失败了怎么办？

说明 Skill 写得不够强硬。改进方法：

1. 在 Step 1 加上（⚠️ 必须执行，不可跳过）
2. 在"禁止行为"中明确写"不要跳过 Step 1"
3. 在 Skill 末尾加上"所有禁止行为都意味着：停下来，从头走流程。"

## Step 7: 迭代优化

**V1**（初版）：AI 跳过了 Step 1

**V2**（加上禁止语句）：

- ```
## ⚠️ 禁止行为
```
- 不要跳过"理解意图"步骤直接给建议

V3（加强制语句）：

```
### Step 1: 理解意图（⚠️ 必须执行，不可跳过）
```

先回答：这段代码要解决什么问题？

V4（测试通过）：AI 每次都按流程走 ✅

完整流程总结

```
Step 1: 确定文件位置
↓
Step 2: 创建 SKILL.md 文件
↓
Step 3: 编写完整内容
↓
```

💡 **记住：**Skill 不是一次写成的，而是通过反复测试逐步优化的。这就是 Loop Engineering（循环工程）的体现。

💡 练习说明：每个练习都是一个真实开发场景。你可以独立完成，也可以让 AI 辅助你撰写 Skill。**重点是学会思考——这个 Skill 应该解决什么问题？流程是什么？怎么写才能让 AI 照着做？**

练习A	练习B	练习C	练习D	练习E
API 文档	单元测试	Bug 报告	Git Commit	数据库优化
(结构化输出)	(代码分析)	(日志解析)	(变更理解)	(性能优化)
★	★★	★★	★★★	★★★★★

Step 1: 读取代码, 识别 `@app.get/post` 装饰器

Step 2: 提取路径、方法、参数定义

Step 3: 生成参数说明表格

- Step 4: 生成示例请求和响应
- Step 5: 输出 Markdown 格式文档

3. 输出格式模板：

```
## POST /api/users

**功能**：创建新用户

### 请求参数

| 参数名 | 类型 | 必填 | 说明 |
|-----|-----|-----|-----|
| username | string | 是 | 用户名，3-20位 |
| email | string | 是 | 邮箱地址 |

### 返回值

```json
{
 "id": 123,
 "username": "zhangsan",
 "created_at": "2024-01-01"
}
```

动手试试：

```

name: api-doc-generator
description: Use when [什么时候触发?]

API 文档生成

Overview
[用1-2句话说明这个Skill做什么]

Core Pattern
[写出你的步骤]

Output Format
[定义输出格式]
```



## 练习 B：单元测试生成 Skill

**场景：**你写了一个函数，需要写单元测试。手动写测试用例太费时间。

**需求：**写一个 Skill，让 AI 读取 Python 函数代码，自动生成 pytest 测试用例。

**引导思路：**

### 1. 想一想：好的测试用例应该覆盖哪些情况？

- 正常情况（Happy Path）
- 边界条件（空值、极大值、极小值）
- 异常情况（错误输入、异常处理）

### 2. 设计 Core Pattern：

```
Step 1: 读取函数签名和实现
Step 2: 识别输入参数的类型和约束
Step 3: 设计正常情况的测试
Step 4: 设计边界条件的测试
Step 5: 设计异常情况的测试
Step 6: 生成 pytest 代码
```

### 3. 示例：

```
原始函数
def calculate_discount(price: float, discount_rate: float) -> float:
 """计算折扣后价格, discount_rate 范围 0-1"""
 if not 0 <= discount_rate <= 1:
 raise ValueError("折扣率必须在0-1之间")
 return price * (1 - discount_rate)

AI 应该生成的测试
def test_calculate_discount_normal():
 assert calculate_discount(100, 0.2) == 80.0

def test_calculate_discount_zero():
 assert calculate_discount(100, 0) == 100.0

def test_calculate_discount_full():
 assert calculate_discount(100, 1) == 0.0

def test_calculate_discount_invalid():
 import pytest
 with pytest.raises(ValueError):
 calculate_discount(100, 1.5)
```

**动手试试：**设计你的 Skill，让 AI 能自动生成这样的测试。

练习 C：Bug 分析报告 Skill

**场景：**测试同事发现了一个 bug，给你发了一堆错误日志。你需要写一份结构化的 bug 报告给开发团队。

**需求：**写一个 Skill，让 AI 分析错误日志，生成结构化的 bug 报告。

**引导思路：**

1. **想一想：**一份好的 bug 报告应该包含哪些信息？

- 问题标题（简洁明了）
- 复现步骤（怎么重现这个 bug）
- 预期行为 vs 实际行为
- 错误日志/截图
- 严重程度评估
- 可能的原因分析

2. **设计 Core Pattern：**

Step 1: 解析错误日志，提取关键信息（异常类型、堆栈、时间戳）  
Step 2: 识别问题发生的模块/组件  
Step 3: 推断复现步骤（从日志反推用户操作）  
Step 4: 对比预期行为和实际行为  
Step 5: 评估严重程度（P0-P3）  
Step 6: 生成结构化报告

3. **输出格式模板：**

```
Bug 报告

标题
[简洁描述问题]

严重程度
● P0 / ● P1 / ● P2 / ● P3

复现步骤
1. ...
2. ...
3. ...

预期行为
[应该发生什么]

实际行为
[实际发生了什么]

错误日志
```

[关键日志片段]

```
可能原因
[基于日志的初步分析]
```

**动手试试：**给 AI 一段错误日志，看它能否生成完整的报告。

---

## 练习 D：Git Commit 消息生成 Skill

**场景：**你改了一堆代码，要提交了。写 commit message 太费脑子，总是写"update code"。

**需求：**写一个 Skill，让 AI 分析代码变更，生成规范的 Git commit message。

**引导思路：**

1. **想一想：**一个好的 commit message 应该包含什么？

- 类型（feat/fix/docs/refactor/test/chore）
- 作用域（改了哪个模块）
- 简洁的摘要（做了什么）
- 详细说明（为什么这么做，可选）

2. **学习 Conventional Commits 规范：**

```
<type>(<scope>): <subject>

<body>
```

示例：

```
feat(auth): 添加JWT token刷新机制

- 实现access token过期后自动刷新
- 添加refresh token的存储和验证
- 解决用户频繁重新登录的问题
```

3. **设计 Core Pattern：**

```
Step 1: 分析 git diff，识别变更的文件
Step 2: 理解每个文件的变更内容（新增/修改/删除）
Step 3: 判断变更类型（新功能？修复？重构？）
Step 4: 确定作用域（哪个模块）
```

Step 5: 生成简洁的摘要（50字以内）

Step 6: 可选：生成详细说明

**动手试试：**让你的 Skill 能根据代码变更自动生成规范的 commit message。

## 练习 E：数据库查询优化 Skill

**场景：**用户反馈某个页面加载很慢。你查了一下，发现是某个 SQL 查询要跑 5 秒。

**需求：**写一个 Skill，让 AI 分析慢 SQL，给出优化建议。

**引导思路：**

### 1. 想一想：SQL 查询慢的常见原因有哪些？

- 没有索引？
- 索引失效？
- 查询了太多数据？
- JOIN 太多表？
- 子查询太复杂？

### 2. 设计 Core Pattern：

Step 1: 分析 SQL 语句结构

Step 2: 识别潜在性能问题（WHERE条件、JOIN、子查询）

Step 3: 检查是否有合适的索引

Step 4: 给出优化建议（添加索引？改写SQL？）

Step 5: 提供优化后的 SQL

Step 6: 解释为什么这样优化

### 3. 示例：

```
-- 原始慢查询
SELECT * FROM orders WHERE DATE(created_at) = '2024-01-01';

-- AI 应该给出的优化建议
-- 问题：DATE()函数导致索引失效
-- 优化：改为范围查询
SELECT * FROM orders
WHERE created_at >= '2024-01-01'
AND created_at < '2024-01-02';
```

**动手试试：**让 AI 能分析 SQL 并给出优化建议。

**练习提示：**怎么写好 Description

不管选哪个练习，记住 description 的写法：

✖ 错误写法	✔ 正确写法
description: 生成API文档	description: Use when need to generate Markdown documentation from FastAPI code
description: 写单元测试	description: Use when writing pytest tests for Python functions
description: 分析bug	description: Use when analyzing error logs and generating bug reports

**记住：**description 回答的是"AI 什么时候应该加载这个 Skill"，不是"这个 Skill 能做什么"。

练习检查清单

写完每个 Skill 后，对照检查：

- ☐ description 以 "Use when" 开头
- ☐ Overview 用 1-2 句话说清核心原则
- ☐ Core Pattern 步骤清晰、可操作
- ☐ 有输出格式模板
- ☐ 实际调用过，验证了效果

九、面试常见问题

Q1：什么是 Skill？和 Prompt 有什么区别？

**标准答案：**

Skill 是封装了提示词、上下文和工具调用规则的 Markdown 文件，让 AI 在特定场景下按固定流程做事。和 Prompt 的区别：

维度	Prompt	Skill
复用性	一次性，用完就丢	可复用，持久保存
结构化	一段文字	有 frontmatter、Overview、Core Pattern 等标准结构
触发方式	手动输入	AI 根据 description 自动判断是否加载
维护成本	每次重写	写一次，持续优化

**通俗理解：**

- Prompt** = 你每次给实习生口头交代任务
- Skill** = 你写好一本《员工手册》，实习生自己看

**加分回答：**

"Skill 是从'单次对话'到'Agent 自主决策'之间的关键一步。它比 Prompt 省力（不用每次重复），比 Agent 简单（不需要 AI 自己判断）。"

Q2：你做过哪些 Skill？举个具体的例子。

标准答案：

我做过以下几个 Skill（举例 2-3 个）：

1. code-review Skill：

- 场景：PR 合并前的代码审查
- 流程：理解意图 → 检查正确性 → 检查可维护性 → 检查性能
- 效果：AI 不再只关注代码风格，而是按三个维度系统化审查

2. test-debugging Skill：

- 场景：测试失败时自动调试
- 流程：读取错误日志 → 定位失败代码 → 分析原因 → 给出修复建议
- 效果：不用每次解释"我们项目用的是 pytest"

3. api-doc-generator Skill：

- 场景：从 FastAPI 代码自动生成接口文档
- 流程：读取代码 → 提取接口定义 → 生成 Markdown 文档

加分回答：

"写 Skill 的关键是从高频重复的工作中找痛点。我每次调试 bug 都要解释项目技术栈，所以写了 test-debugging Skill。每次写完接口都要手动写文档，所以写了 api-doc-generator Skill。"

Q3：Skill 和 Function Calling、MCP 有什么区别？

标准答案：

概念	类比	说明
Function Calling 	单个工具（一把刀）	AI 能调用一个外部函数
Skill 	工具箱 + 操作手册	多个 Function 打包 + 流程编排
MCP 	工具箱接口标准	统一标准让 AI 即插即用各种工具

通俗理解：

- **Function Calling** = 你学会了"切菜"这一招
- **Skill** = 你学会了"做一道完整的菜"——洗菜、切菜、炒菜、摆盘这一整套
- **MCP** = 所有厨具都用统一接口，换一个牌子也能用

加分回答：

"Skill 的核心是'提示词'，不是代码。一个 Skill 本质上是把一个复杂任务拆解成多个步骤，每个步骤可能调用不同的 Function，但 Skill 更关注'流程编排'而不是'单个函数调用'。"

Q4：怎么写好 description？为什么这很重要？

标准答案：

description 的写法规则：

1. 以 **"Use when"** 开头——告诉 AI 什么情况下应该加载这个 Skill
2. 只写触发条件，不写做什么——AI 会偷懒只看 description 不去读正文
3. 不超过 500 字符——太长 AI 不会看
4. 埋入关键词——让 AI 能通过搜索找到这个 Skill

示例：

✗ 错误写法

description: 帮用户调试代码

description: 生成API文档

✓ 正确写法

description: Use when tests fail or code behaves unexpectedly

description: Use when need to generate Markdown documentation from FastAPI code

为什么重要：

description 决定了 AI "什么时候会想到用这个 Skill"。写错了，Skill 就等于不存在。

加分回答：

"这就是 CSO（搜索引擎优化）在 Skill 中的应用。要在 description 和正文中埋入 AI 会搜索的关键词，比如错误信息（test failed、timeout）、症状描述（flaky、hanging）、工具名称（pytest、FastAPI）。"

Q5：怎么防止 AI 跳过 Skill 中的某些步骤？

标准答案：

AI 有时候会偷懒跳过某些步骤。防止方法：

1. 在 Skill 中明确写"禁止行为"：

## ⚠ 禁止行为

- 不要跳过"理解意图"步骤直接给建议
- 不要只看代码风格，忽略逻辑问题

**\*\*所有禁止行为都意味着：停下来，从头走流程。\*\***

2. 在关键步骤加强制语句：

## Step 1: 理解意图 (⚠️ 必须执行，不可跳过)  
先回答：这段代码要解决什么问题？

3. 测试并迭代：如果发现 AI 总是跳过某一步，说明 Skill 写得不够强硬，需要加禁止语句

加分回答：

"渐进式复杂度原则很重要。先写最简版本跑通，发现 AI 跳过步骤后再加禁止语句。不要一开始就写 300 行——越长的 Skill，AI 越容易'选择性阅读'。"

Q6：Skill 有哪些进阶用途？

标准答案：

Skill 不只是"提示词模板"，它有很多进阶用途：

用途	说明	示例
知识沉淀	把团队的最佳实践固化成 Skill	"我们公司的代码规范"Skill
流程标准化	让 AI 按固定流程做事	"代码审查"Skill、"需求分析"Skill
跨项目复用	一个 Skill 在多个项目中使用	"单元测试生成"Skill
经验传承	老员工的经验写成 Skill，新人快速上手	"性能优化"Skill
多 Skill 协作	一个 Skill 引用另一个 Skill	"调试"Skill 引用"日志分析"Skill

加分回答：

"Skill 的本质是'把隐性知识显性化'。很多经验丰富的开发者知道'应该怎么做'，但从来没写下来。Skill 强迫你把知识结构化，不仅 AI 能用，团队其他人也能用。"

Q7：怎么让多个 Skill 协同工作？

标准答案：

方法 1：在一个 Skill 中引用另一个 Skill

**\*\*前置要求\*\***：请先理解 `systematic-debugging` Skill 中的调试流程。

方法 2：设计 Skill 时考虑组合性

- 每个 Skill 只做一件事（单一场景原则）
- Skill 之间通过明确的输入输出接口协作
- 例如："日志分析"Skill 输出结构化日志，"调试"Skill 读取这些日志

方法 3：用 Agent 编排多个 Skill



- Agent 根据任务自动选择合适的 Skill
- 例如：用户说"这个 bug 怎么调试？" → Agent 先调用"日志分析"Skill，再调用"调试"Skill

### 加分回答：

"这就是从 Skill 到 Agent 的演进。Skill 是'你定好规矩，AI 照着做'，Agent 是'AI 自己判断什么时候用什么 Skill'。多个 Skill 协同工作时，要考虑它们之间的数据流和依赖关系。"

Q8：Hermes 等框架会把经验写成可复用的 Skill，它的具体原理是什么？

### 标准答案：

Hermes 是一个 AI Agent 框架，它的核心思想是\*\*"经验学习"\*\*：

1. **执行任务时记录经验**：Agent 完成任务后，把"怎么做的"记录下来
2. **提取可复用的模式**：从多次执行中提取共同的模式和最佳实践
3. **自动生成 Skill**：把模式转化成 Skill 格式，供后续使用

### 类比：

- 就像一个老员工带新员工，不仅告诉他"怎么做"，还把自己多年的经验写成《工作手册》
- 下一次遇到类似问题，新员工直接看手册就行

### 技术原理：

1. **执行追踪**：记录 Agent 的每一步操作（调用了什么工具、输入输出是什么）
2. **模式识别**：分析多次执行，找出共同的模式（比如"每次都先查日志再改代码"）
3. **Skill 生成**：把模式转化成 Skill 格式（frontmatter + Core Pattern + 输出格式）
4. **持续优化**：每次执行后更新 Skill，让 Skill 越来越好

### 加分回答：

"这本质上是'机器学习'的思想——从数据中学习模式。只不过这里学习的是'Agent 的行为模式'，而不是'数据的统计规律'。Hermes 的做法让 Agent 能'越用越聪明'，因为每次执行都在积累经验。"

Q9：AI 是怎么根据 description 决定加载哪个 Skill 的？

### 标准答案：

AI 加载 Skill 的过程：

1. **接收用户请求**：用户说"帮我调试这个 bug"
2. **关键词匹配**：AI 从请求中提取关键词（debug、bug、错误）
3. **搜索 Skill**：AI 在所有 Skill 的 description 和正文中搜索匹配的关键词
4. **选择 Skill**：AI 选择匹配度最高的 Skill 加载
5. **执行 Skill**：AI 按照 Skill 中的 Core Pattern 执行任务

这就是为什么 **description** 要埋入**关键词**（CSO）：

- 如果 description 写"帮用户调试代码"，AI 搜索"debug"时找不到

- 如果 description 写 "Use when tests fail or code behaves unexpectedly", AI 搜索 "fail" "error" "bug" 时都能找到

### 加分回答：

"这就是'搜索引擎优化'（SEO）在 AI 中的应用。要让 Skill 被 AI 找到，就要想清楚'用户会怎么描述这个问题'，然后在 description 中埋入这些描述。"

## Q10：团队中怎么共享和管理 Skill？

### 标准答案：

#### 方法 1：Git 仓库管理

- 所有 Skill 放在团队的 Git 仓库中
- 用 Pull Request 审核 Skill 的变更
- 用 Git 分支管理 Skill 的版本

#### 方法 2：Skill 注册中心

- 建一个中心化的 Skill 仓库（类似 npm）
- 团队成员可以发布和安装 Skill
- 用版本号管理 Skill 的更新

#### 方法 3：按项目组织

- 每个项目有自己的 Skill 目录（`.claude/skills/`）
- 项目级 Skill 只在该项目中生效
- 通用 Skill 放在全局目录（`~/ .claude/skills/`）

### 管理最佳实践：

- **命名规范**：用连字符命名（code-review，不是 codeReview）
- **文档完整**：每个 Skill 都要有 Overview 和 When to Use
- **定期审查**：每月审查一次 Skill，删除过时的、合并重复的
- **质量把控**：新 Skill 必须经过 Code Review 才能合并

### 加分回答：

"Skill 的管理应该像代码一样严格。好的 Skill 是团队的资产，坏的 Skill 会误导 AI。建议用'渐进式发布'：先在个人目录测试，再发布到团队目录，最后发布到全局目录。"

💡 **面试技巧**：回答 Skill 相关问题时，先说"是什么"，再说"怎么做"，最后说"为什么"。面试官喜欢既懂原理又有实践经验的候选人。

## 总结一句话

**Skill 就是把"我教过 AI 无数次的事"写成一份文档，让它自己学会。**

 术语对照索引

本章出现位置	术语	术语手册中对应章节
开篇	<b>Skill</b>	三、Agent 架构与协作
能力阶梯	<b>Function Calling</b>	三、Agent 架构与协作
能力阶梯	<b>Agent</b>	三、Agent 架构与协作
对比表	<b>MCP</b>	三、Agent 架构与协作
设计原则	<b>Prompt Engineering</b>	二、提示与交互
设计原则	<b>Few-shot / Zero-shot</b>	二、提示与交互
设计原则	<b>CoT（思维链）</b>	四、推理范式

学完本章你应该认识的术语

- ☐ **Skill** — 是什么？和 Function Calling 有什么区别？
- ☐ **SKILL.md** — 一个 Skill 文件必须包含哪些部分？
- ☐ **Description** — 怎么写才对？"Use when..."原则是什么？
- ☐ **CSO** — 怎么让 AI 能找到你的 Skill？
- ☐ **设计四原则** — 单一场景、提示词为核心、渐进式复杂度、可测试
- ☐ **防偷懒写法** — 怎么防止 AI 跳过步骤？

附录 A：SKILL.md 模板

```

name: your-skill-name
description: Use when [specific triggering conditions and symptoms]

Your Skill Name

Overview
[核心原则，1-2 句话]

When to Use
- [触发场景 1]
- [触发场景 2]

When NOT to Use
- [不适用场景]

Core Pattern
[核心流程/步骤]

Output Format
[期望的输出格式]
```

```
Quick Reference
| 项目 | 说明 |
|-----|-----|
| ... | ... |

Common Mistakes
- [常见错误 1]
- [常见错误 2]
```

## 附录 B：Skill 质量自检清单

写完 Skill 后，对照这个清单检查：

- ☐ `name` 只用字母、数字、连字符
- ☐ `description` 以 "Use when" 开头
- ☐ `description` 只写触发条件，不写做什么
- ☐ `description` 不超过 500 字符
- ☐ Overview 用 1-2 句话说清核心原则
- ☐ When to Use 列出了具体触发场景
- ☐ 写了 When NOT to Use
- ☐ Core Pattern 步骤清晰、可操作
- ☐ 有输出格式模板
- ☐ 实际调用过，验证了效果
- ☐ 关键词覆盖充分（便于 AI 搜索）